

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to identify the three primary sources of security requirements that drive Secure SDLC adoption in cloud development.
- By the end of this lecture, you will be able to describe the specific security activities that belong in each phase of the Secure SDLC, from requirements through maintenance.
- By the end of this lecture, you will be able to explain why integrating security during the design phase is exponentially cheaper than retrofitting controls after deployment.
- By the end of this lecture, you will be able to articulate the security professional's active role throughout all SDLC phases, not only during testing or final review.
- By the end of this lecture, you will be able to apply Secure SDLC principles to a cloud application development scenario and identify where security activities belong.

Introduction

The Secure Software Development Life Cycle — Secure SDLC — is the structured answer to one of software engineering's most persistent problems: security that arrives too late to be effective. When security reviews happen only at the end of a development cycle, they are analogous to reviewing a bridge's structural blueprints after the bridge has already been built. Problems discovered at that stage are expensive to fix, architecturally disruptive, and frequently produce controls that are retrofitted rather than designed in. The Secure SDLC resolves this by making security a continuous discipline applied at every phase, from the first requirement through ongoing maintenance.

The CCSP exam treats Secure SDLC not as a single methodology but as a philosophy: whatever development process an organisation uses — Waterfall, Agile, hybrid — the Secure SDLC commitment means security requirements, threat modelling, secure coding standards, and security testing are woven into that process at the appropriate phases. The automotive industry provides the clearest analogy: crumple zones and airbag placement are not added to a finished car. They are constraints that shape the design from the first sketches. Security built into a design is more effective, more maintainable, and less expensive than security bolted onto a finished product.

For Cloud Security Practitioners, understanding the Secure SDLC means understanding both the business drivers that make it necessary and the specific security activities that belong in each phase. Regulatory obligations such as GDPR and HIPAA, contractual requirements such as PCI DSS, and organisational risk appetite all generate security requirements that must enter the development process at the requirements phase — not as late corrections. A security professional who waits until testing to raise these concerns has already lost the most cost-effective window for addressing them.

Core Concepts Explained

Secure SDLC as an Engineering Philosophy

Definition: The Secure SDLC is the commitment to integrating security requirements, security decisions, and security verification at every phase of software development, regardless of which development methodology the team uses.

Business relevance: Defects identified during requirements or design cost a fraction of those fixed post-deployment. Industry research consistently shows remediation costs increase by an order of magnitude for each phase the fix is deferred.

Cloud Security Practitioner relevance: A Cloud Security Practitioner is an active participant at every SDLC phase — establishing requirements, reviewing designs, verifying implementations, and monitoring production — not a gatekeeper who reviews the finished product.

Real-world example: An organisation building a multi-tenant SaaS platform on AWS discovered during design-phase threat modelling that their planned data isolation model used shared database schemas with row-level security. The security architect, participating in the design review, identified that a SQL injection vulnerability in any tenant's query path could potentially bypass row-level security and expose other tenants' data. Because this was caught in design, the team restructured to per-tenant isolated schemas — a structural change that would have cost months and required a data migration if caught after deployment. The change took two days at the design stage. Security participation in design prevents problems that cannot be cheaply fixed later.

Risks or limitations: The primary risk of treating Secure SDLC as a final-stage check is architectural debt. Security controls retrofitted onto an insecure design are consistently weaker and more brittle than controls designed in from the start. The best practice that prevents this is formally requiring a security architect review and sign-off before the design phase outputs are baselined.

Business Requirements as the Foundation of Security

Definition: Security requirements derive from three sources: legal and regulatory obligations (GDPR, HIPAA), contractual obligations (PCI DSS, NIST SP 800-53), and organisational risk appetite expressed through risk assessments and business impact analyses.

Business relevance: Treating security requirements as second-class citizens during planning is one of the most reliable predictors of security failures later. Requirements written at the start as testable acceptance criteria are cheaper, clearer, and more defensible than controls specified during implementation.

Cloud Security Practitioner relevance: The Cloud Security Practitioner translates regulatory obligations and risk assessments into concrete, testable security requirements and ensures those requirements enter the product backlog with equal priority to functional requirements.

Real-world example: A healthcare technology company building a patient engagement portal on Microsoft Azure had developers who understood HIPAA at a general level but had not translated that understanding into specific requirements. When a security architect reviewed the backlog before sprint planning, they found no explicit requirement for encryption of data at rest, no requirement for audit logging of record access, and no requirement for session timeout — all HIPAA Security Rule controls. The architect rewrote each obligation as an acceptance criterion: 'All PHI stored in Azure Blob Storage must be encrypted using customer-managed keys in Azure Key Vault.' Developers then built to these requirements rather than discovering the gaps during a pre-launch compliance review that would have required significant rework.

Risks or limitations: The risk of poorly specified security requirements is that they become ambiguous and optional under deadline pressure. The best practice is expressing security requirements in the same format as

functional requirements — specific, testable, and assigned a priority in the backlog — so they cannot be quietly dropped without explicit risk acceptance.

Phase-by-Phase Security Activities

Definition: Each SDLC phase has a specific set of security activities: requirements (capture security requirements as acceptance criteria), design (threat modelling, least privilege architecture), implementation (secure coding standards, peer review, static analysis), testing (functional and adversarial security testing), deployment (configuration review, secrets management), and maintenance (vulnerability management, monitoring, incident response).

Business relevance: Phase-specific security activities create natural checkpoints that prevent vulnerabilities from propagating forward in the development cycle. Each checkpoint is cheaper than discovering the same issue one phase later.

Cloud Security Practitioner relevance: Cloud Security Practitioners are responsible for ensuring each phase's security activities are actually executed — actively joining design reviews, reviewing test results, validating deployment configurations, and participating in post-incident reviews.

Real-world example: A financial services firm adopting a DevSecOps model mapped every Secure SDLC activity to a workflow stage in their GitLab CI/CD pipeline. During implementation, a developer committing code triggered a Semgrep static analysis scan tuned for their Java codebase. During testing, OWASP ZAP ran against the staging environment on every merge to the release branch. During deployment, Checkov validated Terraform infrastructure-as-code against their hardening baseline before any cloud resource was provisioned. During maintenance, AWS Security Hub aggregated findings across all environments. Each phase had an automated gate, and a human security engineer reviewed gate failures that automated tools could not resolve. No SDLC phase could complete without security having been addressed within it.

Risks or limitations: The risk of poorly defined phase activities is security becoming nominal — listed in the SDLC document but not performed. The best practice is requiring a security artefact as output of each phase: a signed-off threat model from design, a static analysis report from implementation, a penetration test summary from testing.

Secure Coding Standards and OWASP ASVS

Definition: The OWASP Application Security Verification Standard (ASVS) provides a comprehensive set of security requirements for how code should be written and how it should behave under attack conditions, used during the implementation phase to define what acceptable code looks like.

Business relevance: Without explicit secure coding standards, developers make independent and inconsistent security decisions. ASVS creates a shared reference that enables consistent peer review, automated tool configuration, and compliance documentation.

Cloud Security Practitioner relevance: Cloud Security Practitioners select and tune ASVS verification levels appropriate to the application's risk profile, configure static analysis tools to enforce ASVS requirements, and participate in code review to evaluate security-relevant findings.

Real-world example: A team building an API gateway for a cloud-native insurance platform adopted OWASP ASVS Level 2 as their implementation standard. They configured SonarQube static analysis rules to map to ASVS categories covering input validation (ASVS 5), authentication (ASVS 2), and session management (ASVS 3). During code review, reviewers used an ASVS-aligned checklist rather than reviewing informally. Within two sprints, the team identified and remediated a JWT token validation flaw that would have allowed an attacker to forge authentication tokens by switching the algorithm to 'none' — a classic ASVS 3.5.3 failure. The ASVS

checklist prompted the reviewer to look specifically at token validation logic, where the flaw was hidden in a third-party library's default configuration.

Risks or limitations: The risk of omitting coding standards is inconsistent security quality across the codebase — secure modules alongside vulnerable ones. The best practice is making ASVS compliance part of the peer review sign-off criteria, so no code is merged without a reviewer explicitly attesting to having checked security-relevant items.

Maintenance Phase and Ongoing Security Posture

Definition: The Secure SDLC does not end at deployment. The maintenance phase includes ongoing vulnerability management (monitoring and patching dependencies), security monitoring (detecting anomalies in production), and incident response (feeding lessons learned back into future development cycles).

Business relevance: Most cloud applications accumulate dependency vulnerabilities over time as CVEs are disclosed against third-party libraries and cloud services. Without a maintenance-phase process, an application that was secure at deployment degrades into a vulnerable system within months.

Cloud Security Practitioner relevance: Cloud Security Practitioners own or participate in the vulnerability management programme, configure and review security monitoring dashboards, conduct post-incident reviews, and ensure lessons learned produce updated requirements in the next development cycle.

Real-world example: A team running Node.js microservices on Google Cloud Platform integrated Dependabot into their GitHub repositories, which automatically opened pull requests when a dependency's CVE was published. Security engineers triaged Dependabot alerts weekly, prioritising by CVSS score and whether the vulnerable code path was exercised in production. When Log4Shell (CVE-2021-44228) was disclosed in December 2021, teams with automated dependency scanning were able to identify affected services within hours. The team had a complete affected-services list and a patch deployment plan within 90 minutes of the CVE becoming public — compared to manual inventory processes that took other organisations days.

Risks or limitations: The risk of neglecting maintenance-phase security is dependency vulnerability accumulation — a well-designed application slowly becoming exploitable as its dependencies age. The best practice is treating dependency updates as a recurring sprint story with a defined SLA based on CVSS severity: critical within 48 hours, high within seven days, medium within the sprint cycle.

Practical Applications

The Secure SDLC principles described in this lecture apply directly to the decisions Cloud Security Practitioners make in the earliest stages of any development engagement.

Establishing Security Requirements Before Sprint Planning

A Cloud Security Practitioner joining a new cloud application project participates in the pre-sprint requirements workshop rather than waiting to review a finished design. They work through the regulatory landscape — identifying whether the application will handle PII subject to GDPR, financial data subject to PCI DSS, or health information subject to HIPAA — and translate each applicable obligation into acceptance criteria written in the same format as functional user stories. They also conduct a preliminary risk assessment to identify the organisation's risk appetite for this application class, converting the assessment into non-functional security requirements: data encryption requirements, authentication strength requirements, session management requirements. The practitioner's decision rule: every security requirement must be specific (what must be true), testable (how it will be verified), and prioritised (what happens if deadline pressure requires a trade-off). Vague requirements like 'the application should be secure' are rewritten before the backlog is finalised. Requirements

enter the sprint planning backlog alongside functional stories, with defined acceptance criteria that developers can implement and QA can verify.

Leading a Design-Phase Threat Modelling Session

During the design phase of a cloud data pipeline project, a Cloud Security Practitioner leads a STRIDE-based threat modelling session with the lead developer and cloud architect. The team works through the system's data flow diagram, examining each trust boundary: the ingestion API receiving data from external partners, the transformation layer processing that data in a managed cloud service, the storage tier writing to an encrypted data warehouse, and the analytics layer exposing results to internal users. For each boundary, the practitioner asks the STRIDE questions systematically: who could spoof the identity of an external partner? What could an attacker tamper with in the transformation layer? Which actions lack sufficient audit logging for repudiation protection? The session produces a prioritised list of architectural decisions — specifically, that partner API clients must authenticate with mutual TLS rather than API keys, and that the transformation layer must run in a VPC with no public internet egress. Both decisions are captured in the design document before development begins, eliminating the need to retrofit network controls after the infrastructure is provisioned.

Running a Maintenance-Phase Vulnerability Management Programme

A Cloud Security Practitioner responsible for a deployed cloud application establishes a maintenance-phase monitoring programme integrated with the development team's sprint cycle. They configure AWS GuardDuty and CloudTrail to alert on anomalous API call patterns — targeting credential stuffing signatures and unusual IAM role assumption events. They subscribe to CVE feeds for every runtime dependency and create a weekly triage process: CVSS 9+ patches within 48 hours, high severity within seven days, medium severity within the sprint cycle. When a vulnerability is discovered in a third-party authentication library, the practitioner's triage confirms the vulnerable code path is exercised in production, raises a P1 issue in the team's backlog, and pairs with the lead developer to test the patched version in staging before the end-of-day deployment window. The practitioner also chairs a post-incident review after every security event, ensuring lessons identified produce backlog items that enter the next development cycle's requirements phase — closing the loop between maintenance and future development.

Best Practices

- **Capture security requirements alongside functional requirements.** Security requirements should enter the product backlog in the same format as functional user stories, written as testable acceptance criteria: 'All API responses containing user data must be transmitted over TLS 1.2 or higher.' Requirements written this way are testable, assignable, and difficult to quietly drop under deadline pressure without an explicit risk acceptance decision.
- **Require a security artefact as output of every SDLC phase.** A signed-off threat model from design, a static analysis report from implementation, a penetration test summary from testing, and a deployment hardening checklist from deployment create an audit trail demonstrating security was addressed in each phase. In regulated environments, this documentation is required by compliance auditors looking for phase-specific evidence.
- **Never treat the maintenance phase as optional.** Configure automated dependency scanning for every production repository and define a documented SLA for patch deployment based on CVSS severity. Applications that were secure at deployment will accumulate critical vulnerabilities within 12 to 18 months without active dependency management and a process for acting on what scanning discovers.

- **Engage security professionals in design review, not only in testing.** The most expensive place to catch a security flaw is a penetration test after deployment. The cheapest is a design review before implementation begins. Security practitioners should attend design sessions with a prepared STRIDE-based question set and the explicit mandate to raise architectural concerns while changes are still cheap to make.
- **Protect non-negotiable security requirements from deadline pressure.** Establish at project initiation which security requirements are non-negotiable — regulatory or contractual obligations that cannot be deferred without explicit risk acceptance from the business owner and a documented exception. This prevents the common pattern of security requirements being silently dropped when timelines compress.

Common Mistakes and Misconceptions

Misconception: Treating security as a final-phase gate — commissioning a penetration test only in the sprint before launch.

Correct approach: Security testing in the final sprint catches real vulnerabilities but cannot fix architectural flaws cheaply. The best practice is integrating security testing throughout the SDLC: static analysis in implementation, functional security testing in testing, and penetration testing as confirmation rather than primary discovery.

Misconception: Writing security requirements as vague preferences — 'the application should follow security best practices' — rather than testable acceptance criteria.

Correct approach: Every security requirement should be specific and verifiable. 'The application must enforce a minimum password length of 12 characters and reject passwords appearing in the HIBP breach corpus' is a testable requirement. 'The application should have strong passwords' is not and will be interpreted differently by different developers.

Misconception: Treating the Secure SDLC as a developer responsibility, with security professionals only reviewing finished work.

Correct approach: Security professionals are most effective as active participants from the requirements phase onward. A security architect who attends only the final test review has missed every opportunity to influence design decisions while they were cheap to change.

Misconception: Assuming deployment is the end of the Secure SDLC.

Correct approach: The maintenance phase is where many cloud applications fail — not because they were poorly built but because their dependencies aged without patching. The best practice is establishing a vulnerability management programme before the application launches, with automated scanning and a defined SLA for remediation.

checklist: Secure SDLC Phase Gate Checklist

Use this checklist to verify that security activities have been completed before the work of each SDLC phase is baselined. Each gate should be confirmed by the Cloud Security Practitioner before the next phase begins.

- ☐ Requirements: Security requirements identified from regulatory obligations (GDPR, HIPAA, PCI DSS)
- ☐ Requirements: Security requirements identified from contractual obligations (NIST SP 800-53, SLAs)
- ☐ Requirements: Organisational risk appetite documented and translated into security requirements

- ☐ Requirements: All security requirements written as testable acceptance criteria in the backlog
- ☐ Design: Threat modelling session completed using STRIDE or equivalent framework
- ☐ Design: Least privilege applied to all data flows, permissions, and service integrations
- ☐ Design: Threat model document baselined and stored in the project artefact repository
- ☐ Implementation: Secure coding standard (OWASP ASVS or equivalent) adopted and communicated
- ☐ Implementation: Static analysis tool configured and running in CI pipeline
- ☐ Implementation: Peer code review includes a security-focused checklist
- ☐ Testing: Functional security tests verify all security acceptance criteria pass
- ☐ Testing: Dynamic analysis tool (OWASP ZAP or equivalent) executed against staging environment
- ☐ Testing: Penetration test scoped and completed (if required by risk profile)
- ☐ Deployment: Default credentials changed; unnecessary services disabled
- ☐ Deployment: Secrets injected via secrets manager, not embedded in configuration files
- ☐ Deployment: Change management record confirms deployed artefacts match tested and approved versions
- ☐ Maintenance: Automated dependency scanning configured with defined SLA for patch deployment
- ☐ Maintenance: Security monitoring dashboards configured and alert thresholds set
- ☐ Maintenance: Post-incident review process defined and communicated to the team

How to use this: *In Agile environments, apply the relevant checklist items to each sprint based on which phase activities that sprint encompasses. Not every sprint touches every phase, but every sprint introducing new functionality should trigger design and implementation gates.*

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A development team is building a cloud application to process European customer data. The security architect argues that GDPR requirements should be captured during the requirements phase. A project manager responds that the compliance team will handle GDPR documentation separately after development. Which of the following best describes the risk of the project manager's approach?

- A) A. GDPR requirements are only needed for the audit trail and do not affect application design.
- B) B. Handling GDPR requirements post-development may require expensive architectural changes to implement data protection controls that should have been designed in from the start.
- C) C. GDPR compliance is the responsibility of the legal team, not the development team.
- D) D. Deferring GDPR requirements to post-development is standard practice in Agile projects.

Answer: B. Handling GDPR requirements post-development may require expensive architectural changes to implement data protection controls that should have been designed in from the start.

Q2. Which OWASP standard is specifically used during the implementation phase of the Secure SDLC to define what acceptable secure code looks like?

- A) A. OWASP Top Ten

- B) B. OWASP Application Security Verification Standard (ASVS)
- C) C. OWASP Software Assurance Maturity Model (SAMM)
- D) D. OWASP Testing Guide

Answer: B. OWASP Application Security Verification Standard (ASVS)

Q3. In a Secure SDLC, which phase is most cost-effective for identifying and addressing architectural security flaws?

- A) A. Testing phase, because security testing tools can find vulnerabilities automatically.
- B) B. Deployment phase, because production configuration reviews reveal real-world risks.
- C) C. Design phase, because architectural decisions are still flexible and changes are cheap to make.
- D) D. Maintenance phase, because production monitoring reveals actual attack patterns.

Answer: C. Design phase, because architectural decisions are still flexible and changes are cheap to make.

Q4. A Cloud Security Practitioner reviews a cloud application's security posture 18 months after deployment. Despite being well-designed at launch, the application now has several known critical vulnerabilities. No architectural changes have been made since launch. What is the most likely cause?

- A) A. The initial threat model was incomplete.
- B) B. Third-party library dependencies have accumulated CVEs that were not patched during the maintenance phase.
- C) C. The deployment configuration was not hardened adequately.
- D) D. The testing phase did not include penetration testing.

Answer: B. Third-party library dependencies have accumulated CVEs that were not patched during the maintenance phase.

Q5. A security requirement states: 'The application should follow security best practices for authentication.' Why is this requirement inadequate, and how should it be improved?

- A) A. It is inadequate because authentication is the responsibility of the cloud provider, not the development team.
- B) B. It is adequate; security best practices are well-understood and do not need to be specified further.
- C) C. It is inadequate because it is vague and untestable; it should be rewritten as specific, measurable acceptance criteria such as 'Users must authenticate using a minimum of two factors; passwords must be at least 12 characters and pass HIBP breach corpus checks.'
- D) D. It is inadequate because authentication requirements belong in the testing phase, not the requirements phase.

Answer: C. It is inadequate because it is vague and untestable; it should be rewritten as specific, measurable acceptance criteria such as 'Users must authenticate using a minimum of two factors; passwords must be at least 12 characters and pass HIBP breach corpus checks.'

Reflection Questions

1. Consider an application you are familiar with — either one you have worked on or one you use professionally. At what point in its development do you believe security requirements were first formally considered? What would have changed if those requirements had been established at the beginning?

2. The lecture uses the automotive crumple zone analogy to explain why security designed in outperforms security bolted on. Can you identify another engineering domain where the same principle applies? What does that comparison tell you about the universality of the Secure SDLC philosophy?
3. If you were joining a development team midway through a project that had not adopted a Secure SDLC, what single phase activity would you prioritise introducing first, and why?
4. The maintenance phase is frequently deprioritised after launch. What organisational factors contribute to this tendency, and what governance mechanisms could counteract them?
5. OWASP ASVS provides three verification levels. How would you select the appropriate level for a public-facing cloud application that handles payment card data, and what criteria would inform that decision?

Key Takeaways

- The Secure SDLC is a philosophy, not a methodology: it means integrating security into whatever development process the organisation uses, at every phase from requirements through maintenance.
- Security requirements come from three sources: regulatory obligations (GDPR, HIPAA), contractual obligations (PCI DSS, NIST SP 800-53), and organisational risk appetite — and must enter the backlog as testable acceptance criteria.
- Each SDLC phase has specific security activities: requirements capture, design-phase threat modelling, implementation-phase secure coding standards, testing-phase adversarial testing, deployment-phase configuration review, and maintenance-phase vulnerability management.
- The cost of fixing a security flaw increases by an order of magnitude for each phase it is deferred. Design-phase fixes are the cheapest; post-deployment fixes are the most expensive.
- OWASP ASVS is the primary implementation-phase standard for defining what acceptable secure code looks like, enabling consistent peer review and automated tool configuration.
- The Cloud Security Practitioner's role is active participation throughout all SDLC phases — not final-stage review. The most valuable security work happens before the first line of code is written.
- The maintenance phase is where most deployed cloud applications fail over time. Automated dependency scanning with a defined patch SLA is the minimum maintenance-phase security control.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Review your current development team's backlog and identify whether security requirements are written as testable acceptance criteria or as vague preferences. Rewrite at least three requirements into specific, verifiable acceptance criteria format this week.
7. Map your organisation's current SDLC phases against the Secure SDLC phase checklist in this guide. Identify which phases currently have no defined security artefact output and propose one concrete deliverable for each gap.
8. Enable automated dependency scanning on at least one production repository this week, document the SLA for patch deployment based on CVSS severity, and present the results and process to your team in the next sprint retrospective.
9. Identify one upcoming design review on your team's calendar and request to attend with the explicit mandate of conducting a STRIDE walkthrough on the proposed data flow diagram.